Student 7 **Jayadeep J** 192511304RC 4 / 4 submitted

SIMATS Engineering • CSE • CSA0802 — Python Programming • 29-07-2026 • Category: Scenario Based Program Writing • Student Submissions Report

Q1. Intelligent Customer Feedback Analysis

CODE

```
def normalize_text(text):
    """
    Normalize feedback by:
    - converting to lowercase
    - removing unwanted characters (keep only alphabets, digits, and spaces)
    - collapsing multiple spaces
    """
    text = text.lower().strip()

    # Replace punctuation/undesired characters with space
    cleaned = []
    for ch in text:
        if ch.isalnum() or ch.isspace():
            cleaned.append(ch)
        else:
            cleaned.append(' ')
    cleaned_text = ''.join(cleaned)

    # Collapse multiple spaces
    parts = cleaned_text.split()
    return " ".join(parts)


def count_keywords(text, keywords):
    """
    Count occurrences of each keyword in the text.
    Uses loop + conditionals.
    """
    tokens = text.split()
    count = 0

    for token in tokens:
        if token in keywords:
            count += 1

    return count


def classify_sentiment(pos_count, neg_count):
    """
    Sentiment classification using if-elif-else.
    """
    if pos_count > neg_count:
        return "Positive"
    elif neg_count > pos_count:
        return "Negative"
    else:
        return "Neutral"


def analyze_feedback(feedback, pos_keywords, neg_keywords):
    """
    Modular function to preprocess, count keywords, and classify.
    """
    normalized = normalize_text(feedback)
    pos_count = count_keywords(normalized, pos_keywords)
    neg_count = count_keywords(normalized, neg_keywords)

    sentiment = classify_sentiment(pos_count, neg_count)
    return pos_count, neg_count, sentiment


def main():
    # You can edit keywords for better accuracy
    positive_keywords = {"good", "great", "excellent", "awesome", "nice", "love", "happy", "satisfied", "fantastic"}
    negative_keywords = {"bad", "poor", "terrible", "awful", "hate", "worst", "sad", "unsatisfied", "broken"}
```

```
    n = int(input("Enter number of feedback entries: "))

    results = []
    for i in range(n):
        feedback = input(f"Enter feedback {i+1}: ")
        pos_count, neg_count, sentiment = analyze_feedback(feedback, positive_keywords, negative_keywords)
        results.append((pos_count, neg_count, sentiment))

    print("\n--- Sentiment Analysis Results ---")
    for idx, (pos_count, neg_count, sentiment) in enumerate(results, start=1):
        print(f"Feedback {idx}: Positive={pos_count}, Negative={neg_count} → {sentiment}")


if __name__ == "__main__":
    main()
```

OUTPUT

```
(no output)
```

SIMATS Engineering • CSE • CSA0802 — Python Programming • 29-07-2026 • Category: Scenario Based Program Writing • Student Submissions Report

Q2. Comprehensive Password Strength Checker

CODE

```
def g(p):
    a = len(p)

    if a == 0:
        return ("Weak", "Empty password. Please enter a valid password.")

    if a < 6:
        return ("Weak", "Too short. Use at least 8 characters for better strength.")

    u = l = d = s = 0
    for ch in p:
        if 'A' ≤ ch ≤ 'Z':
            u = 1
        elif 'a' ≤ ch ≤ 'z':
            l = 1
        elif '0' ≤ ch ≤ '9':
            d = 1
        else:
            s = 1

    b = p.lower()
    pred1 = (b == "password")
    pred2 = (b == "123456")
    pred3 = ("qwerty" in b)
    rep = 0

    for i in range(a - 1):
        if p[i] == p[i + 1]:
            rep += 1

    m = 0
    if u == 0: m += 1
    if l == 0: m += 1
    if d == 0: m += 1
    if s == 0: m += 1

    score = 0
    score += (a ≥ 10) * 2
    score += (a ≥ 8) * 1
    score += u + l + d + s

    if pred1 or pred2:
        score -= 3
    if pred3:
        score -= 2
    if rep ≥ 2:
        score -= 2
    if m ≥ 2:
        score -= 2

    if score ≤ 2:
        t = "Weak"
    elif score ≤ 5:
        t = "Moderate"
    elif score ≤ 8:
        t = "Strong"
    else:
        t = "Very Strong"

    f = []
    if u == 0:
        f.append("add uppercase letters (A-Z)")
    if l == 0:
        f.append("add lowercase letters (a-z)")
    if d == 0:
        f.append("add digits (0-9)")
    if s == 0:
```

```
        f.append("add special symbols (e.g. !@#$)")
    if a < 8:
        f.append("increase length to at least 8 characters")
    if pred1:
        f.append("avoid using the word 'password'")
    if pred2:
        f.append("avoid using '123456' type patterns")
    if pred3:
        f.append("avoid keyboard patterns like qwerty")
    if rep ≥ 2:
        f.append("avoid too much repetition of the same character")

    if len(f) == 0:
        f = ["looks good! keep it unique and avoid reusing passwords"]

    msg = "Strength: " + t + ". Feedback: " + ", ".join(f)
    return (t, msg)


def main():
    print("═══ CO2_AT2 - Password Strength Checker ═══")
    p = input("Enter password: ")

    t, msg = g(p)

    print("\nResult:")
    print(msg)


if __name__ == "__main__":
    main()
```

OUTPUT

```
(no output)
```

SIMATS Engineering • CSE • CSA0802 — Python Programming • 29-07-2026 • Category: Scenario Based Program Writing • Student Submissions Report

Q3. Structured Server Log Error Analyser

CODE

```
def ext(s):
    a = []
    for line in s:
        a.append(line.strip())
    return a

def get_err(a):
    b = []
    for c in a:
        if ("ERROR" in c) or c.startswith("ERROR"):
            b.append(c)
        elif ("Error" in c) or c.startswith("Error"):
            b.append(c)
    return b

def cate(l):
    if "404" in l or "Not Found" in l:
        return "404"
    elif "500" in l or "Internal Server" in l:
        return "500"
    elif "TIMEOUT" in l or "timed out" in l.lower() or "timeout" in l.lower():
        return "TIMEOUT"
    elif "AUTH" in l or "unauthorized" in l.lower() or "forbidden" in l.lower():
        return "AUTH"
    else:
        if l.find("ERROR") =/= -1:
            return "OTHER"
        return "OTHER"

def dedup(a):

    b = []
    seen = set()
    for c in a:
        if c not in seen:
            seen.add(c)
            b.append(c)
    return b

def summary(a):
    c = {}
    for d in a:
        k = cate(d)
        if k in c:
            c[k] += 1
        else:
            c[k] = 1
    return c

def report(a, c):
    print("\n--- Error Analysis Report ---")
    print("Total Error Lines (after duplicate removal):", len(a))
    print("\nCategory Counts:")
    for k in ["404", "500", "TIMEOUT", "AUTH", "OTHER"]:
        if k in c:
            print(k, "⇒", c[k])
    for k in c:
        if k not in ["404", "500", "TIMEOUT", "AUTH", "OTHER"]:
            print(k, "⇒", c[k])

    print("\nUnique Error Entries:")
    for e in a:
        print("-", e)

def main():
    print("Paste log lines. Type 'END' on a new line to finish.\n")
    a = []
```

```
    while True:
        x = input()
        if x.strip() == "END":
            break
        a.append(x)

    a = ext(a)
    b = get_err(a)
    d = dedup(b)
    c = summary(d)
    report(d, c)

if __name__ == "__main__":
    main()
```

OUTPUT

```
(no output)
```

SIMATS Engineering • CSE • CSA0802 — Python Programming • 29-07-2026 • Category: Scenario Based Program Writing • Student Submissions Report

Q4. Email Domain Frequency Analyser

CODE

```
def ext(e, a, b):
    if '@' not in e:
        return

    parts = e.split('@')
    if len(parts) ≠ 2:
        return

    d = parts[1].strip().lower()
    if d == '' or '.' not in d:
        return

    if d not in b:
        a.append(d)
        b.add(d)


def solve(emails)
    c = []
    f = set()
    e = []
    for x in emails:
        before = len(c)
        ext(x, c, f)
        if len(c) > before:
            e.append(0)
    for x in emails:
        if '@' not in x:
            continue
        p = x.split('@')
        if len(p) ≠ 2:
            continue

        dom = p[1].strip().lower()
        if dom == '' or '.' not in dom:
            continue
        for i in range(len(c)):
            if c[i] == dom:
                e[i] += 1
                break
    if len(c) == 0:
        return None, 0, {}
    mx = e[0]
    k = c[0]
    for i in range(len(e)):
        if e[i] > mx:
            mx = e[i]
            k = c[i]
    out = {}
    for i in range(len(c)):
        out[c[i]] = e[i]
    return k, mx, out
print("CO2_AT2 - Email Domain Frequency Analyser")
n = int(input("Enter number of emails: ").strip())
arr = []
for i in range(n):
    arr.append(input(f"Email {i+1}: ").strip())
top_dom, top_cnt, all_cnt = solve(arr)
if top_dom is None:
    print("No valid email domains found.")
else:
    print("\nDomain Frequencies:")
    for dom in all_cnt:
        print(dom, "→", all_cnt[dom])
```

```
    print("\nMost Common Domain:")
    print(top_dom, "→", top_cnt)
```

OUTPUT

```
(no output)
```

Student 8 **JILLALAMUDI SAI MADHURI** 192312204RC 0 / 4 submitted

Q1. Intelligent Customer Feedback Analysis

— Not Submitted —

Q2. Comprehensive Password Strength Checker

— Not Submitted —

Q3. Structured Server Log Error Analyser

— Not Submitted —

Q4. Email Domain Frequency Analyser

— Not Submitted —